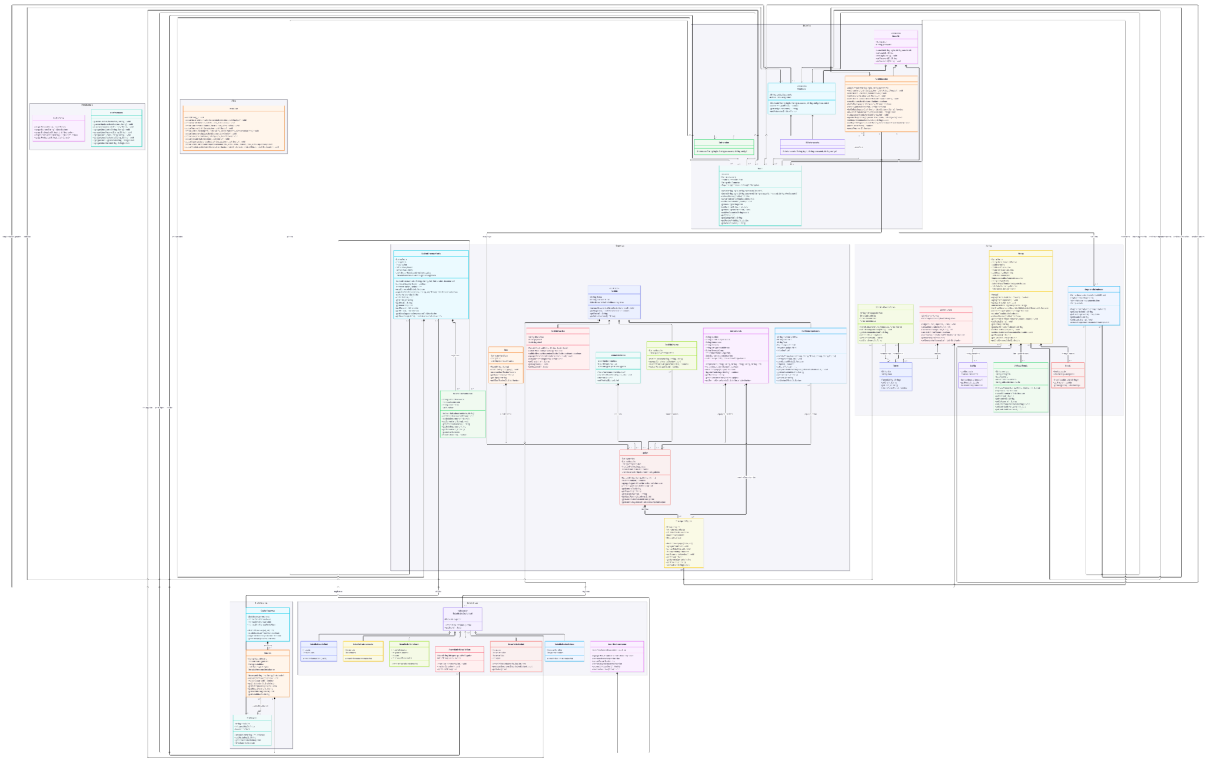
Documentación Entrega 2 Proyecto 1

Cristian Sneider Monroy 202522919
Carlos Espinosa 202515673
Simon Arteta 202523759

a. Diagrama de clases de diseño



Justificación de las 7 principales decisiones de diseño:

1. Uso de abstracción y herencia

Se utilizaron clases abstractas como Usuario, Empleado, Partido y EstadisticaIndividual para establecer estructuras comunes y permitir especializaciones. Esta decisión favorece la reutilización de código, evita duplicaciones y facilita la aplicación del polimorfismo.

2. Separación de responsabilidades administrativas

La clase Administrador concentra las operaciones administrativas, mientras que Turno y SolicitudCambioTurno representan la asignación de horarios y las solicitudes de modificación. Esta separación permite organizar las responsabilidades sin concentrar toda la información administrativa en una sola clase.

3. Especialización de partidos y estadísticas

Los partidos se especializan en internos, competitivos y de práctica. Las estadísticas se separan según la disciplina deportiva, permitiendo registrar los indicadores correspondientes a cada deporte sin mezclar información incompatible.

4. Separación de equipos, inscripciones y competencias

Se utilizaron las clases Equipo, InscripcionEquipo y Competencia para diferenciar la organización de los equipos, la participación de los socios y la inscripción en torneos. Esta estructura permite gestionar cada proceso de manera independiente y conservar sus relaciones.

5. Separación de entrenamientos y asistencia

Se diferenciaron SesionEntrenamiento y AsistenciaEvaluacion para almacenar por separado la información general de cada entrenamiento y los registros individuales de asistencia y desempeño de los participantes.

6. Organización de reservas y ventas

Las clases Reserva y Venta representan procesos independientes. Sus relaciones permiten identificar los participantes, instalaciones y productos involucrados, manteniendo separada la información deportiva y comercial.

7. Relaciones y multiplicidades

Se utilizaron herencia, asociaciones, agregaciones y composiciones para representar las relaciones entre entidades. Las multiplicidades establecen las cantidades permitidas de objetos relacionados, mientras que las composiciones representan vínculos estructurales entre elementos, como sesiones y registros de asistencia.

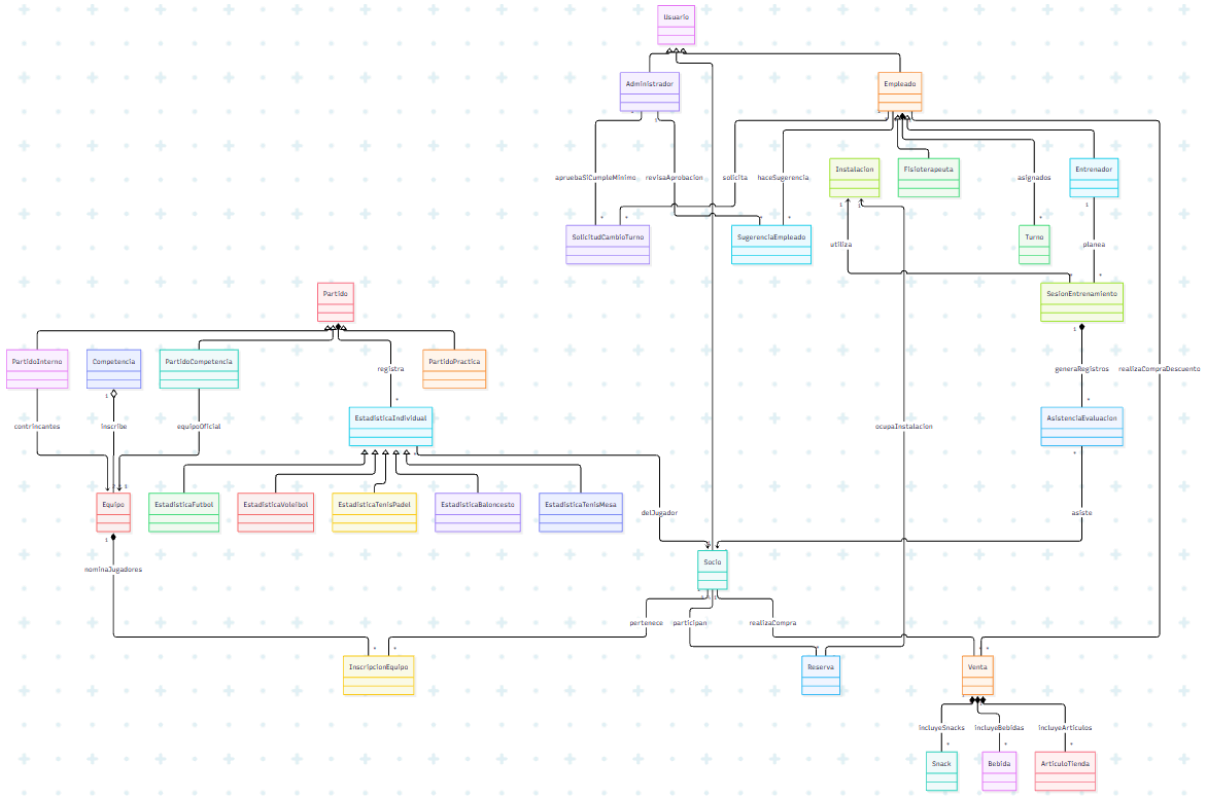
En conjunto, estas decisiones permiten mantener un diseño organizado, modular y coherente con los principios de programación orientada a objetos, facilitando su implementación en Java.

b. Diagrama de clases de alto nivel

El diagrama de clases de alto nivel presenta la estructura general del sistema de gestión del Club Deportivo Séneca, identificando sus principales componentes y las relaciones existentes entre ellos.

Este modelo se deriva del diagrama de clases de diseño detallado y conserva sus clases, junto con las relaciones de herencia, asociación y composición representadas. Se omiten los atributos y métodos para facilitar la comprensión de la arquitectura orientada a objetos.

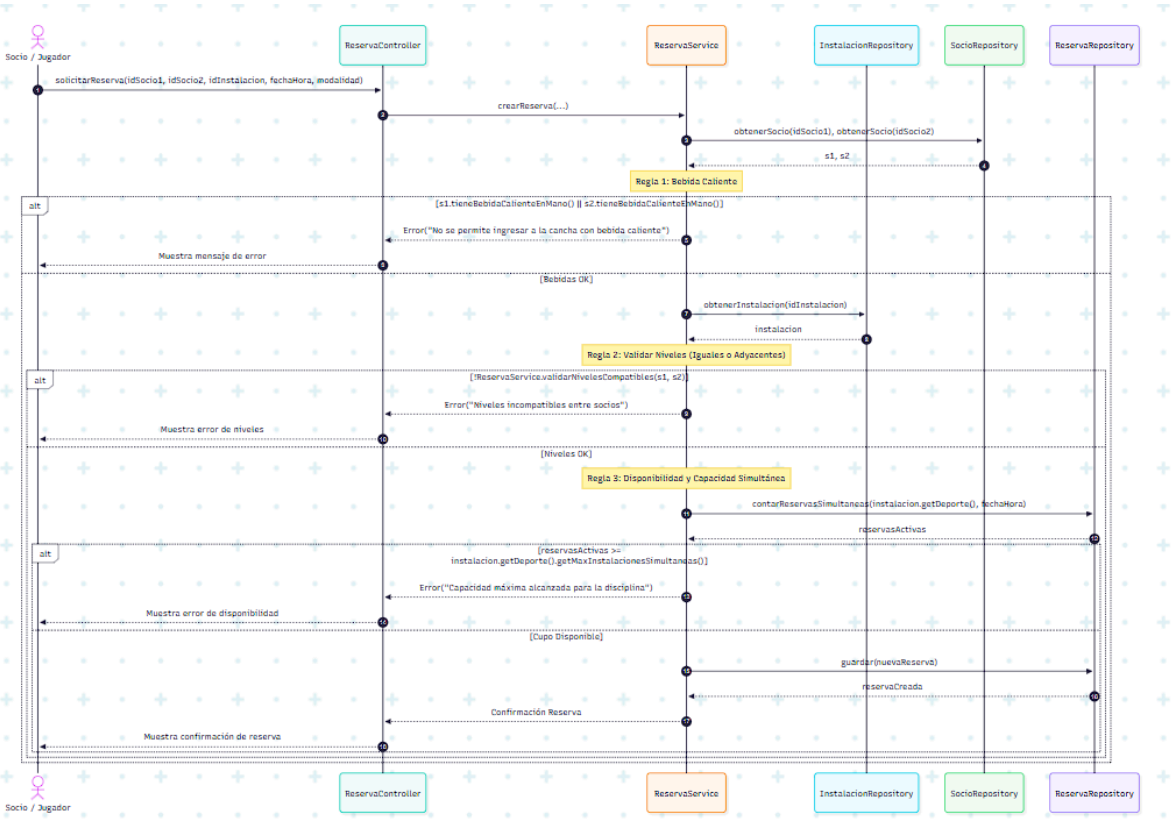
Su objetivo es proporcionar una visión global del sistema, permitiendo identificar las responsabilidades generales de las clases, sus dependencias y la organización de los distintos módulos funcionales.



c. Diagramas de secuencia para las funcionalidades críticas

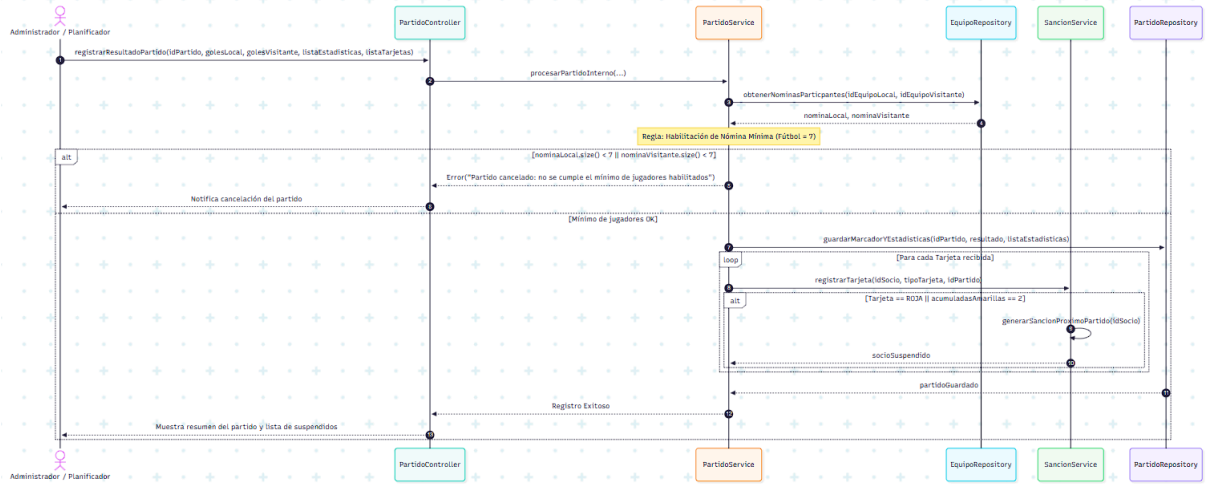
1. **Reserva de Instalación de Raqueta**

Es una funcionalidad crítica porque entrelaza la disponibilidad física, el nivel deportivo y las normas de seguridad del club. El sistema debe validar que los socios tengan niveles compatibles para garantizar partidos equitativos, verificar la capacidad máxima por disciplina para evitar el sobrecupo, y consultar en tiempo real el estado de bebida caliente del socio para denegar el acceso a canchas restringidas por razones de seguridad física.



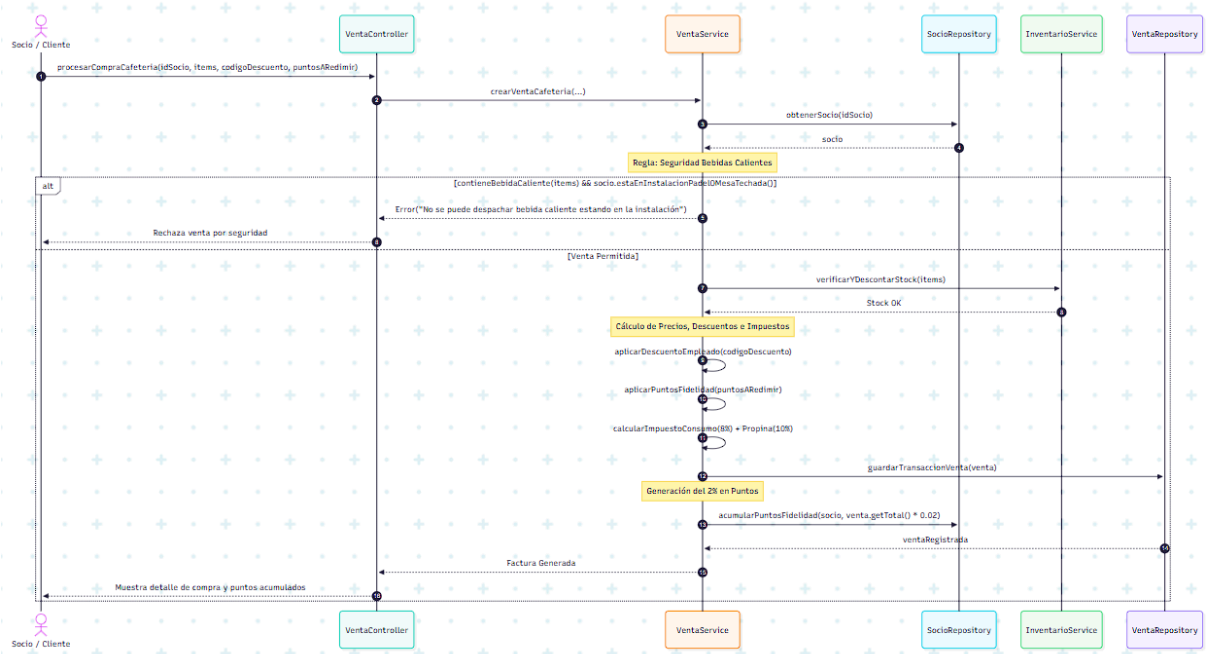
2. **Registro de Partido de Fútbol Interno y Suspensión por Tarjetas**

Es fundamental para la disciplina y la validez de los encuentros deportivos. La lógica de negocio no solo debe certificar que ambos equipos cumplan con la nómina mínima de jugadores habilitados antes de procesar el juego, sino que debe calcular de forma automática las sanciones por acumulación de tarjetas amarillas o rojas para actualizar el estado de suspensión del deportista de cara a la siguiente fecha oficial.



3. Venta en Cafetería

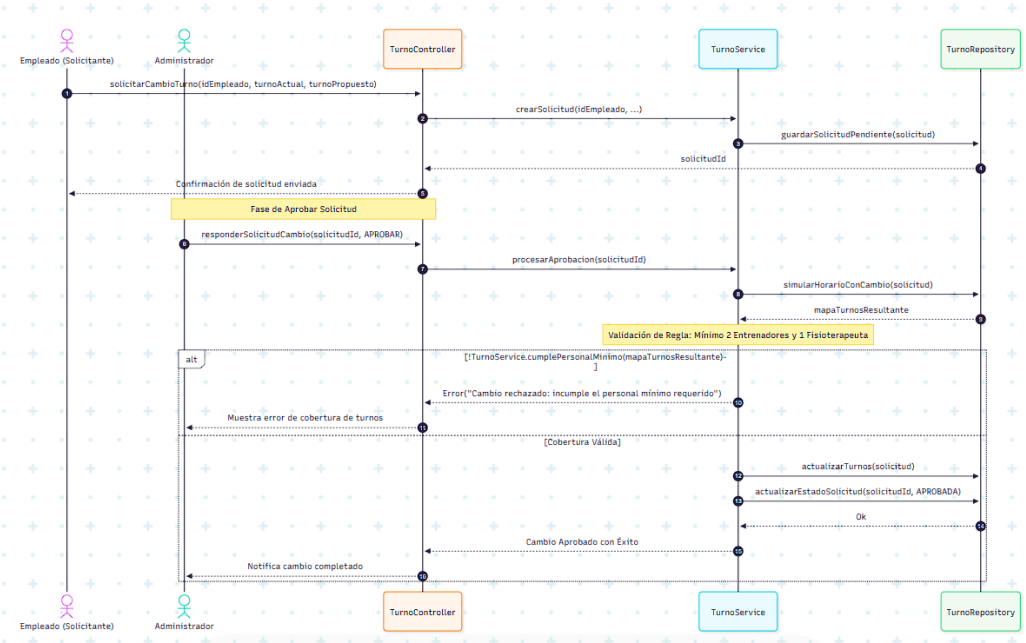
Destaca por su alta complejidad financiera, impositiva y de seguridad física. El flujo debe calcular correctamente el impuesto al consumo, la propina y los descuentos de empleados no acumulables, acumular el 2% en puntos de fidelidad y, al mismo tiempo, realizar una validación cruzada con la ubicación del socio para impedir el despacho de bebidas calientes si se encuentra dentro de canchas de pádel o mesas de tenis de mesa techadas.



4. Solicitud y Aprobación de Cambio de Turno de Empleados

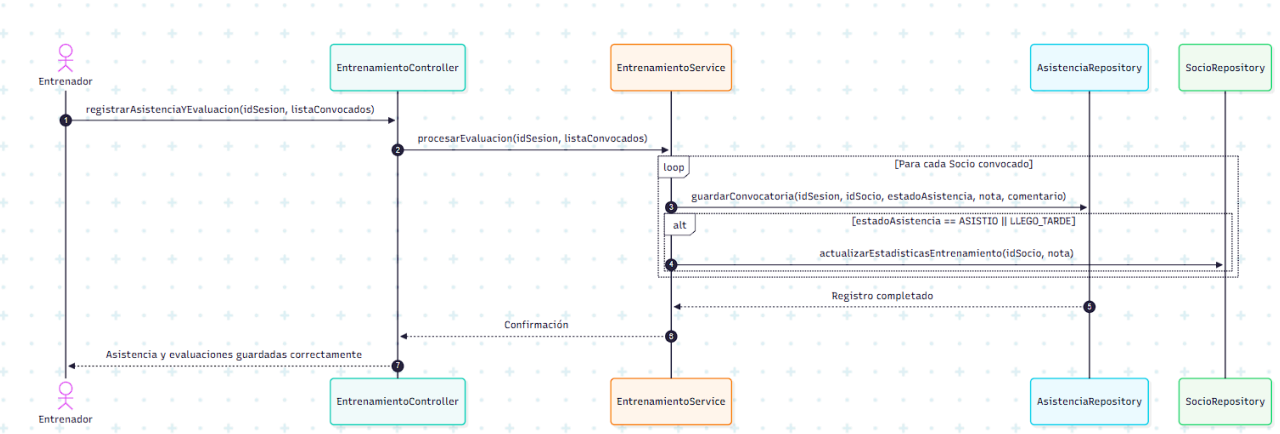
Es crítica para garantizar la continuidad operativa y legal de la sede. El sistema no puede limitarse a cambiar un estado, sino que debe simular algorítmicamente la cobertura horaria

resultante y bloquear automáticamente cualquier permuta que incumpla la norma innegociable del club de mantener siempre disponibles como mínimo a dos entrenadores y un fisioterapeuta durante la jornada.



5. Evaluación de Desempeño y Asistencia en Sesión de Entrenamiento

Constituye la fuente primaria de datos para todo el módulo de analítica deportiva. Al registrar de forma persistente la asistencia y las calificaciones cualitativas y cuantitativas de los atletas, el sistema alimenta de manera precisa los reportes de evolución semestral, la toma de decisiones para convocatorias oficiales y el seguimiento continuo de los deportistas de la institución.



D. Organización y responsabilidades del sistema

El sistema se divide en las siguientes áreas:

Área	Responsabilidad
Usuarios	Identificar socios, empleados y administrador, y controlar sus permisos.
Gestión deportiva	Administrar equipos, inscripciones, partidos, competencias, estadísticas y sanciones.
Reservas	Controlar disponibilidad, capacidad, modalidad y uso de instalaciones.
Entrenamientos	Organizar sesiones y registrar asistencia y evaluaciones.
Gestión comercial	Administrar inventario, ventas, descuentos, impuestos y puntos de fidelidad.
Administración	Gestionar turnos, solicitudes, sugerencias y reportes.
Persistencia	Guardar y recuperar la información del sistema.

1. Sistema y Persistencia

ClubDeportivoSeneca: Almacena y gestiona las colecciones de todo el sistema (usuarios, equipos, competencias, instalaciones, reservas, partidos, entrenamientos y ventas). Coordina la ejecución de las operaciones de la aplicación.

GestorPersistencia: Lee y escribe la información del sistema en los archivos planos ubicados en la carpeta externa configurada.

2. Usuarios y Personal

Usuario: Define los atributos de autenticación (login y password) comunes para todos los roles del sistema.

Socio: Almacena edad, nivel de raqueta y ranking ELO. Acumula el 2% del valor de cada compra en puntos de fidelidad y gestiona su redención, 1 punto = 1 peso.

Empleado: Almacena el código de descuento corporativo (15% propio, 8% transferible) y mantiene la lista de turnos asignados.

Entrenador: Planea sesiones de entrenamiento y actúa como excepción para permitir partidos de práctica entre socios de niveles no adyacentes.

Fisioterapeuta: Representa al personal médico requerido para cumplir el mínimo de presencia física exigido en la sede.

Administrador: Gestiona altas y bajas de equipos y socios, traslada inventario entre ubicaciones, reabastece artículos, aprueba sugerencias de empleados, genera reportes de ventas y deportivos, y aprueba solicitudes de cambio de turno.

Turno: Define el día y franja horaria laboral asignada a un empleado.

SolicitudCambioTurno: Registra la petición de cambio de horario de un empleado y su estado de aprobación, validando que siempre haya al menos 2 entrenadores y 1 fisioterapeuta en servicio.

3. Equipos y Competencias

Equipo: Administra la lista de jugadores inscritos, valida la edad según la categoría (Sub-13, Sub-15, Sub-17, Mayores), controla el cupo máximo de inscripciones y contabiliza los jugadores habilitados.

InscripcionEquipo: Asocia un socio a un equipo específico, asignando su posición, validando número de camiseta único y controlando amonestaciones y suspensión.

Competencia: Gestiona la inscripción de equipos en torneos externos (ASCUN o Cerros), valida requisitos de nómina mínima y actualiza la tabla de posiciones con los puntos obtenidos.

4. Partidos y Estadísticas

Partido: Modela los atributos base de un encuentro deportivo (fecha, marcador final y estadísticas individuales asociadas).

PartidoInterno: Valida que ambos equipos del club cumplan con el mínimo de jugadores habilitados según el deporte (fútbol: 7, baloncesto: 5, voleibol: 6) para permitir disputar el juego.

PartidoCompetencia: Registra la institución rival, escenario y fase de torneo. Calcula los puntos asignados según el resultado y deporte (fútbol: 3 victoria, 1 empate, 0 derrota; baloncesto y voleibol: define ganador).

PartidoPractica: Valida que los dos socios participantes tengan niveles iguales o adyacentes, salvo que uno de ellos sea entrenador.

EstadisticaIndividual: Asocia el rendimiento de un socio a un partido determinado.

EstadisticaFutbol: Registra goles, asistencias, tarjetas amarillas y rojas. Aplica la suspensión automática al acumular 2 amarillas o recibir 1 roja.

EstadisticaBaloncesto: Registra puntos y rebotes.

EstadisticaVoleibol: Registra aces y bloqueos.

EstadisticaTenisPadel: Registra sets ganados, games ganados, aces y errores no forzados.

EstadisticaTenisMesa: Registra sets ganados y puntos totales.

5. Instalaciones, Reservas y Entrenamiento

Instalacion: Almacena el tipo de disciplina, capacidad máxima y si la cancha o mesa es techada.

Reserva: Controla el bloque horario, cantidad de jugadores y valida la capacidad según la modalidad, pádel: 4 en dobles; tenis y tenis de mesa: 2 en sencillos o 4 en dobles.

SesionEntrenamiento: Registra fecha, hora, duración, entrenador a cargo, instalación utilizada y lista de socios convocados.

AsistenciaEvaluacion: Registra el estado de asistencia de cada convocado (asistió, faltó o llegó tarde), un comentario opcional y la nota de desempeño restringida al rango de 1 a 5.

6. Tienda y Cafetería

ArticuloTienda: Almacena nombre, categoría, precio, cantidad disponible en inventario y ubicación física.

Bebida: Almacena precio e indica si la bebida es fría o caliente.

Snack: Almacena precio y mantiene el registro de alérgenos (maní, gluten, lácteos, etc.).

Venta: Liquida la transacción aplicando el 19% de IVA a la tienda, el 8% de impuesto al consumo a la cafetería, los descuentos , la propina del 10% y la asignación del 2% en puntos de fidelidad. Valida que no se despachen bebidas calientes a socios con reservas activas en pádel o tenis de mesa techado.

SugerenciaEmpleado: Almacena la descripción de nuevos productos propuestos por los empleados para evaluación del administrador.

Persistencia

Para cumplir con el requisito de que toda la información del sistema sea persistente y se almacene en archivos dentro de una carpeta separada del código fuente, se optó por usar archivos planos de texto con separador punto y coma (;). Esta decisión se tomó porque los archivos de texto son legibles, fáciles de inspeccionar y depurar, no requieren dependencias externas y funcionan únicamente con las clases estándar de Java (BufferedReader, FileReader, BufferedWriter y FileWriter), lo que mantiene el proyecto liviano y portable. Además, el formato es definido por el equipo, y el separador ; es adecuado porque ninguno de los atributos del dominio lo contiene.

La carpeta data/ se creó al mismo nivel que src/, no dentro de ella, para cumplir con la restricción de que la carpeta de persistencia no puede ser la misma del código fuente. Esto separa los datos del código, evita que una limpieza del proyecto borre la información y facilita la entrega, ya que la carpeta viaja junto con el proyecto y el README.

La persistencia se dividió en múltiples archivos, uno por tipo de entidad, para evitar archivos monolíticos difíciles de mantener y para reflejar el modelo de dominio. Los archivos son: socios.txt, equipos.txt, empleados.txt, instalaciones.txt, productos.txt, nominas.txt, ventas.txt, reservas.txt, sesiones.txt, asistencias.txt, sugerencias.txt y turnos.txt. Cada archivo corresponde a una entidad o agregado del diagrama de clases, lo que permite escrituras independientes y lectura selectiva.

Se distinguen dos estrategias de escritura según la naturaleza del dato. Para entidades cuyo estado actual debe reflejarse tal cual, como socios, equipos y nóminas, se usa sobrescritura completa con FileWriter sin append, de modo que el archivo siempre refleje el estado vigente y se eviten duplicados. Para entidades que representan historial o eventos que no deben perderse, como ventas, reservas, sesiones, asistencias, sugerencias y turnos, se usa append con FileWriter en modo append, de modo que cada evento se agrega al final y se preserva la trazabilidad para construir reportes acumulados.

Las responsabilidades se separaron en dos clases dentro del paquete persistencia. LectorDatos contiene los métodos estáticos cargarSocios, cargarEquipos, cargarEmpleados, cargarInstalaciones y cargarProductos, que abren el archivo, leen línea por línea, separan por ; y construyen los objetos del dominio, capturando los errores de entrada/salida con try-with-resources sin detener la aplicación. EscritorDatos contiene los métodos guardarSocios, guardarEquipos y guardarNominas para sobrescritura, y los métodos agregarLineaVenta, agregarLineaReserva, agregarLineaSesion, agregarLineaAsistencia, agregarLineaSugerencia y agregarLineaTurno para append. Cada método recibe la ruta del archivo como parámetro, lo que permite cambiar la ubicación sin modificar la lógica. Esta separación cumple con el principio de responsabilidad única: la lógica de negocio no sabe cómo se persiste; solo LectorDatos y EscritorDatos se encargan de ello.

La persistencia se integra en el Principal en tres momentos. Al arrancar, se llama a LectorDatos para cargar socios, equipos, empleados, instalaciones y productos desde data/, reconstruyendo el estado inicial del club. Durante las pruebas, se ejecutan las operaciones sobre los objetos en memoria. Al finalizar, se llama a EscritorDatos.guardarSocios, guardarEquipos y guardarNominas para reflejar el estado actualizado, y a los métodos agregarLinea para registrar los eventos generados.

Como limitación, la lectura al arrancar solo cubre socios, equipos, empleados, instalaciones y productos. Para el Proyecto 2 se planea leer también nóminas, ventas, reservas, sesiones, asistencias, sugerencias y turnos, validar la integridad de los archivos y, si el volumen de datos crece, migrar a un formato estructurado como JSON o CSV con cabecera, manteniendo la misma separación por entidad.

Justificación de la arquitectura por paquetes

El código se organizó en paquetes por dominio: Usuarios, Deportes, Estadisticas, Instalaciones, Ventas, Persistencia y View. Esta separación sigue el principio de alta cohesión y bajo acoplamiento: cada paquete agrupa clases que cambian por las mismas razones. Por ejemplo, todas las clases de ventas (ArticuloTienda, Bebida, Snack, Venta, Turno, SugerenciaEmpleado, GestorTurnos) están juntas porque comparten la lógica de tienda y cafetería. Esto facilita la navegación del código, permite que varios desarrolladores trabajen en paralelo sin conflictos y hace explícitas las dependencias entre módulos: View depende de todos, Persistencia depende de las entidades, y las entidades no dependen de View ni de Persistencia.